C++ PROGRAMMING – ASSIGNMENT 2

CO2: Functions and Object-Oriented Programming | Total: 50 Marks

Student Name: _____

Register Number: _____

Department: _____

Submission Date: _____

C++ PROGRAMMING – ASSIGNMENT 2

CO2: Functions and Object-Oriented Programming

Total Marks: 50

Student Name: _____

Register Number: _____

Department: _____

Submission Date: _____

Table of Contents

Introduction

Q1 – Call by Value, Reference and Pointer

Q2 – Return by Reference

Q3 – Default Arguments and Overloading

Q4 – Class with Internal Array

Q5 – Static Members and Arrays of Objects

Q6 – Friend and Inline Functions

Q7 – Design Challenge

Conclusion

Introduction

This report presents implementations and evaluations for seven C++ tasks covering functions and object-oriented programming. The programs use C++17 and standard library facilities. Each task includes design reasoning, predicted behaviour, implementation notes and complexity analysis.

Q1 – Call by Value, Reference and Pointer

Prediction for the supplied program: change(a,b,&c) changes only b and c, producing 1 12 13. bigger(a,c)=100 assigns through a returned reference to the larger variable; since c is 13 and a is 1, c becomes 100, giving 1 100. show(1), show(1,2), show(1,2,3) print 16, 18 and 6. Combined line: 16 13 6. By-value parameters are local copies; reference parameters alias the caller; pointer parameters hold an address and mutate via dereference. TODO expression: fee -= fee * percent / 100. Expected fee output is 45000 then 40500. A const reference is appropriate for large read-only objects. Returning a new fee is often clearer for pure calculation; in-place mutation is suitable when update semantics are explicit. O(1) time in each arithmetic version. A long long copy is constant-size; copying a large Student scales with its data, while references avoid that copy.

Q2 – Return by Reference

The maximum element is found in O(n), O(1) auxiliary space. Returning int& allows highestMark(marks,n) += 2. A const int& allows read-only access; returning int creates a copy. Returning a local variable by reference creates a dangling reference and undefined behaviour; compile with warnings and do not execute/use it. A vector may reallocate during growth, invalidating pointers/references/iterators. An index is generally safer across reallocation if the same logical element remains; reacquire a pointer/reference after growth.

Q3 – Default Arguments and Overloading

The original overload set is ambiguous because multiple overloads can accept one argument. The implementation separates grams (int), kilograms (double with trailing defaults), and Parcel (const reference). Predictions: charge(2) selects int exact match; charge(2.0) selects double exact match; charge(2.0f) selects double by floating promotion; calls with boolean flags select the double overload; a Parcel selects the Parcel overload. Exact match outranks promotion, which outranks conversion. Defaults fill omitted trailing arguments and must be specified from the right. Overload resolution is compile-time; const Parcel& avoids copying.

Q4 – Class with Internal Array

SavingsAccount encapsulates account number, balance, ten transaction entries, count and next circular insertion index. Public deposit, withdraw and miniStatement methods preserve invariants. The 11th transaction overwrites the oldest. Shifting costs O(k) per full-buffer insertion; circular insertion costs O(1), both use O(k) storage. A deque supports efficient end operations but has dynamic-storage overhead. Increasing capacity to 50 requires changing capacity and display logic only.

Q5 – Static Members and Arrays of Objects

The shared issued counter belongs to the class and is defined outside it. The static count function has no this pointer and cannot access an individual object's non-static fields. The Token program predicts `101 103 104 4`: three array constructors then one extra constructor; issued() is 4. The BusTicket array gives Asha Seat 1, Bala Seat 2, Chitra Seat 3 and Dev Waiting, then Issued: 3. A static counter persists across function calls. A TicketOffice object is more suitable when multiple buses need independent capacities/counters. O(1) time and one counter's storage.

Q6 – Friend and Inline Functions

The supplied Time program predicts `205 false`. 09:45 is 585 minutes after midnight; 13:10 is 790; difference is 205. Arrival is not earlier than departure. Equal times produce 0 and false; 11:59 to 12:01 gives 2 and true for earlier(beforeNoon,afterNoon). Friend functions are non-members with access to private data, useful for symmetric two-object operations. Inline is a request, not a guarantee; modern compiler optimisation controls substitution. Run the included benchmark at -O0 and -O2 and record local values; times vary by machine.

Q7 – Design Challenge

Chosen limitation: Q4 shifting transaction history. When full, every insertion shifts k-1 entries, O(k). Objective: preserve the newest k transactions with O(1) insertion. Redesign: circular buffer stores next write position; write and advance modulo k. It uses the same O(k) memory and adds index-order logic. The included Q7 program inserts two million values into both implementations, prints checksums and timings. Matching checksums validate functional equivalence. Record local timings; do not claim universal speedup from a single run. Shifting remains reasonable for tiny capacities and simpler ordered storage.

Conclusion

The tasks demonstrate how parameter passing, references, overload resolution, encapsulation, static data, friend functions and inline functions affect correctness and design. The Q7 redesign illustrates an evidence-based trade-off: a circular buffer reduces insertion work while retaining bounded storage, at the cost of wraparound indexing.

Appendix – Repository

GitHub URL: [Paste your repository URL after uploading]

Compiler: [Enter compiler name and version from g++ --version]

Build flags: -std=c++17 -Wall -Wextra -pedantic; optimisation benchmarks use -O0 and -O2.